Machine Learning Lab Assignment No. 7

Program: M.Tech – 1st Year Course: Machine Learning Type: Practical / Lab Assignment

Total Questions: 3 (Section A: 1, Section B: 1, Section C: 1) Date: 4 November 2026

General Instructions

- Use Python with NumPy, Pandas, Matplotlib, and scikit-learn. Compute every performance measure from its formula using NumPy first, then verify it with the corresponding scikit-learn function wherever the question asks for it.
- For reproducibility, use random_state = 42 for all data splits and models, n_init = 10 for K-Means, and np.random.default_rng(42) for all random draws in Question 3. Report the Python and package versions used.
- Every figure must have a title, labelled axes, and a legend where more than one curve is plotted.
- Add a short "Observations / Inference" note (2-3 sentences) after each question interpreting your results.
- All three questions are compulsory.
- Copied code or shared solutions between students will be treated as plagiarism.

Section A: Supervised Learning Measures (Question 1)

Question 1: Classification and Regression Performance Measures

Dataset / Setup (Classification): Breast Cancer Wisconsin dataset (load_breast_cancer; 569 samples, 30 features), with malignant (target = 0) as the positive class. Use a 70/30 train–test split (stratify = y, random_state = 42), standardize with StandardScaler fitted on the training set only, and train LogisticRegression(max_iter = 1000, random_state = 42). Predict malignant when its predicted probability is at least 0.5.

Dataset / Setup (Regression): Diabetes dataset (load_diabetes; 442 samples, 10 features). Use an 80/20 train–test split (random_state = 42) and train LinearRegression().

Tasks

(a) On the classification test set, report the confusion matrix (TP, FP, FN, TN) and compute accuracy, precision, recall, specificity, and F1-score from these counts; verify accuracy, precision, recall, and F1-score with scikit-learn.

(b) Using the predicted probability of the malignant class, plot the ROC curve together with the diagonal chance line, and compute the AUC; verify it with roc_auc_score.

(c) On the regression test set, compute MAE, MSE, RMSE, and R^2 from their formulas, verify them with scikit-learn, and plot actual versus predicted values with the y = x line.

Section B: Unsupervised Learning Measures (Question 2)

Question 2: Internal and External Clustering Measures on the Iris Dataset

Dataset / Setup: Iris dataset (150 flowers, 4 measurements, 3 species), standardized with StandardScaler. Use the species label only for external evaluation. Cluster with K-Means (k = 3, n_init = 10, random_state = 42) using Euclidean distance.

Tasks

(a) Compute the silhouette score, Davies–Bouldin index, and Calinski–Harabasz index for the clustering, and state for each whether a higher or a lower value is better.

(b) Compute the Adjusted Rand Index, Normalized Mutual Information, and purity against the species labels (purity = sum over clusters of the largest species count in the cluster, divided by 150).

(c) Run K-Means for k = 2 to 6, plot the silhouette score and the Davies–Bouldin index against k in two subplots, and select the best k from these two measures; state whether it matches the true number of species.

Section C: Reinforcement Learning Measures (Question 3)

Question 3: Q-Learning Performance Measures on a Grid World

Dataset / Setup (Environment): 5 × 5 grid; a state is (row, column) with both indexed 0–4, and the state index is 5 × row + column. The start is (0, 0), the goal is (4, 4), and the pits are (1, 3) and (3, 1). Actions: 0 = up, 1 = down, 2 = left, 3 = right; a move off the grid leaves the agent in its cell. Reward: +10 on entering the goal, −10 on entering a pit, and −1 for every other move. The goal and the pits are terminal, and an episode also ends after 100 steps.

Dataset / Setup (Q-Learning): Tabular Q-learning with Q initialized to zeros, $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.1$, and 1000 episodes. Create rng = np.random.default_rng(42) for each training run. At every step draw u = rng.random(); if $u < \varepsilon$ take the action rng.integers(4), otherwise take np.argmax(Q[s]). The update target is r at a terminal transition and $r + \gamma \cdot \max Q(s', \cdot)$ otherwise. The return of an episode is the undiscounted sum of its rewards, and an episode is a success if it ends in the goal. The optimal return is 3 (8 steps).

Tasks

(a) Implement the environment and Q-learning, record the return and the number of steps of every episode, and plot the episode return together with its 50-episode moving average against the episode number.

(b) Compute the success rate and the average return over the first 100 and the last 100 episodes; the convergence episode (the first episode $e \geq 50$ whose mean return over episodes $e - 49$ to e is at least 0); and the cumulative regret, the sum of (3 − return) over all episodes. Plot the cumulative regret against the episode number.

(c) Evaluate the learned greedy policy ($\varepsilon = 0$) from the start state and compare its return and number of steps with the optimal values. Compare it with a uniformly random policy (rng = np.random.default_rng(42), action = rng.integers(4), 1000 episodes of up to 100 steps) using average return and success rate. Then train again with $\varepsilon = 0.3$ and compare it with $\varepsilon = 0.1$ using the last-100-episode success rate and the cumulative regret.